

進階軟體工程 (Advanced Software Engineering)

核心觀念自學測驗題庫 (Chapters 1–3)

「不聞不若聞之，聞之不若見之，見之不若知之，知之不若行之。」——《荀子·儒效》„

📖 3 大核心章節

🎯 31 道經典觀念題

💡 即時檢測與詳解

⚡ 自主節奏隨選作答

知識地圖與章節單元導覽 (Table of Contents)

點選下方任一章節卡片即可直接跳轉作答；亦可於各題頁面點選右上角 *Header* 隨時展開題項目錄。

第一章

軟體工程導論

軟體危機、IEEE 定義、核心活動、布魯克斯法則、職責分離、ISO 25010、倫理與 AI 挑戰。

- 題數：第 01 題 ~ 第 09 題 (共 9 題)
- 核心目標：建立軟體工程思維基石

開始第一章測驗 →

第二章

軟體開發流程

瀑布模型、V-模型、MVP、敏捷宣言、看板 WIP、Scrum、技術債、結對編程與 CI/CD。

- 題數：第 10 題 ~ 第 23 題 (共 14 題)
- 核心目標：掌握敏捷流程與工程實踐

開始第二章測驗 →

第三章

需求工程

使用者 vs 系統需求、領域需求、可量化 NFR、五個為什麼、內隱知識、使用案例與 AI 幻覺。

- 題數：第 24 題 ~ 第 31 題 (共 8 題)
- 核心目標：需求獲取、規格化與驗證

開始第三章測驗 →

第一章 軟體工程導論

基礎觀念、本質複雜度、專業倫理與 AI 衝擊

「軟體危機的根本原因，在於我們嘗試建造超越人類智力掌握極限的巨大軟體系統。」—— 艾茲赫爾·戴克斯特拉 (Edsger W. Dijkstra)

”

Ch 1. 軟體工程導論 · 第 01 題

單元：軟體危機與複雜度

為什麼 1968 年的「軟體危機」無法單純透過購買運算速度更快、記憶體容量更大的電腦硬體來解決？

- A 1960 年代末期電腦硬體製造與記憶體製程完全停滯，無法提供實質的運算效能提升。
- B 危機本質上是系統規模與智力管理帶來的組織複雜度挑戰，更強大的硬體反而刺激並放大了系統規模。**
- C 當時的高階程式語言完全缺乏數學計算基底與編譯器記憶體動態配置能力。
- D 早期的真空管與主機架構在物理特性上無法支援多終端機與遠端通訊網絡的共用協定。

Ch 1. 軟體工程導論 · 第 02 題

單元：IEEE 軟體定義與範疇

依據 IEEE 對「軟體 (Software)」的標準定義，下列何者不屬於軟體的範疇？

- A 可執行的電腦程式與原始碼檔案。
- B 系統資料庫綱要與組態設定檔。
- C CPU 運算處理器與實體記憶體硬體模組。**
- D 軟體安裝步驟與自動化維運部署程序。

Ch 1. 軟體工程導論 · 第 03 題

單元：軟體工程通用核心活動

下列哪一組選項正確地將具體的軟體工程實踐行為，對應到其所屬的四大通用核心活動？

- A** 與利害關係人進行訪談以梳理使用者故事 $\rightarrow$ 軟體規格說明 (Specification)
- B** 撰寫自動化單元測試以模擬資料庫回應 $\rightarrow$ 軟體設計與實作 (Design & Implementation)
- C** 重構資料庫綱要以提升查詢執行效能 $\rightarrow$ 軟體驗證 (Validation)
- D** 將老舊的第三方支付 API 抽換為全新開道服務 $\rightarrow$ 軟體規格說明 (Specification)

Ch 1. 軟體工程導論 · 第 04 題

單元：布魯克斯法則與團隊規模

某個軟體專案目前落後進度 3 週，距離正式交付期限僅剩 2 週。專案經理決定緊急招募 4 位初階程式設計師加入以期加速進度。根據「布魯克斯法則 (Brooks's Law)」，最可能的結果是什麼？

- A 專案將因此提前 1 週順利交付。
- B 專案進度將進一步嚴重延宕，因為資深工程師必須耗費寶貴時間來指導與培訓新人。**
- C 既有開發人員的產出效率將立刻提升一倍。
- D 團隊內部的溝通複雜度將維持不變。

Ch 1. 軟體工程導論 · 第 05 題

單元：單一職責原則 (SRP)

一個電商「訂單處理模組」同時直接處理 HTTP 請求解析、執行信用卡交易、執行 SQL 資料庫查詢，並動態組裝 HTML 收據郵件。此設計最嚴重違反了哪項核心設計原則？

- A** 關注點分離 (Separation of Concerns)：多項互不相干的業務職責緊密糾纏在單一模組中。
- B** YAGNI 原則：在客戶尚未提出明確商業需求前，過早實作臆測的未來擴充功能。
- C** 布魯克斯法則 (Brooks's Law)：對該訂單模組增加工程人手導致團隊溝通成本呈指數上升。
- D** 預應變更 (Anticipation of Change)：系統設定與業務規則被硬編碼寫死在編譯二進位檔中。

Ch 1. 軟體工程導論 · 第 06 題

單元：ISO 25010 軟體品質特徵

下列哪一個選項正確地將真實世界發生的軟體問題，對應到其所屬的 ISO 25010 品質特徵？

- A 資料庫查詢需要耗費 15 秒才能返回結果 $\rightarrow$ 可維護性 (Maintainability · 可測試性)
- B 當第三方外部 API 離線時，整個系統瞬間全面崩潰停擺 $\rightarrow$ 可靠性 (Reliability · 容錯性)**
- C 由於模組間強烈緊密耦合，開發者極難為其撰寫單元測試 $\rightarrow$ 可移植性 (Portability · 適應性)
- D 未加密的 Session Cookie 導致惡意攻擊者輕易竊取身分登入他人帳號 $\rightarrow$ 易用性 (Usability · 易操作性)

Ch 1. 軟體工程導論 · 第 07 題

單元：ACM/IEEE 軟體工程倫理守則

依據 ACM/IEEE 軟體工程倫理守則，若雇主或主管強烈指示工程師撰寫一段偽造安全合規報告、隱匿系統危險的演算法時，工程師的最高倫理義務為何？

- A 服從指示實作，因為雇主支付了工程師薪資，商業利益高於一切。
- B 堅定拒絕並向上呈報，因為保護公眾利益與社會安全的責任絕對高於對雇主的盲目忠誠。**
- C 照常撰寫該段偽造邏輯，但刻意省略所有程式碼文檔與註解。
- D 將該段具爭議的代碼偷偷外包給第三方廠商執行以規避內部責任。

Ch 1. 軟體工程導論 · 第 08 題

單元：AI 編程與代碼流失率 (Code Churn)

在評估 AI 輔助開發的實證研究中（如 GitClear 分析 1.5 億行代碼），「程式碼流失率 (Code Churn)」是關鍵品質指標。高 Code Churn 在 AI 開發環境中反映出何種核心問題？

- A 新提交的程式碼在極短時間內就被頻繁修改、刪除或推翻重寫，反映出 AI 代碼看似產出快速但本質脆弱且未經深思。
- B 編譯器與打包工具在持續部署管線中，主動且有效率地從發布二進位檔案中自動剔除未引用的死碼。
- C 軟體開發團隊因為具備多語言自動轉譯工具，而在專案中頻繁更換底層核心程式語言與技術架構。
- D 自動化測試案例執行速度過快，導致雲端 CI/CD 伺服器的虛擬機器運算配額在短時間內迅速耗盡。

Ch 1. 軟體工程導論 · 第 09 題

單元：AI 驗證與同溫層測試陷阱

工程師請 AI 生成一套複雜的計費結帳模組，並在未提供形式化規格契約下，直接請同一個 AI 自動生成單元測試。測試全部綠燈通過。此時最大的工程風險為何？

- A 同溫層盲點驗證：AI 生成的測試僅是在驗證自身有缺陷的錯誤假設，無法證明符合真實的業務規格。
- B 執行效能瓶頸：AI 生成的測試斷言在執行期需要耗費比人類手寫測試多出數十倍的記憶體運算資源。
- C 編譯語法失敗：既有的主流自動化測試框架無法成功解析大型語言模型所合成的 Mock 假資料結構。
- D 技術版本鎖定：該測試套件會與特定單一雲端供應商的執行時期環境產生強烈且無法抽換的緊密耦合。

第二章 軟體開發流程

流程模型、敏捷哲學、極限編程與工程紀律

「沒有任何單一的技術或管理突破，能在十年內讓軟體的生產力、可靠度或簡易性提升一個數量級。」
—— 佛瑞德·布魯克斯 (Fred Brooks)

”

Ch 2. 軟體開發流程 · 第 10 題

單元：傳統瀑布模型的本質缺點

傳統瀑布模型在實務運作上最主要的核心缺點是什麼？

- ☐ A 為外部法規合規性與稽核所產生的工程文檔過度匱乏。
- ☒ B 一旦專案啟動進行，極難且成本高昂去因應與調適變更的需求。
- ☐ C 在執行過程中完全消除了元件測試與系統整合測試的需求。
- ☐ D 無法在跨國多據點的大型軟體工程組織中進行團隊協同。

Ch 2. 軟體開發流程 · 第 11 題

單元：V-模型與測試左移

相較於傳統瀑布模型，V-模型在工程實務上最主要的核心優勢是什麼？

- ☐ A 在為期兩週的短週期衝刺疊代中持續交付可運行軟體增量。
- ☒ B 強制在早期的規格分析階段，即同步進行測試規劃與驗收準則設計。
- ☐ C 完全消除了對詳細架構設計文件與介面合約的依賴。
- ☐ D 允許客戶在實作編程階段隨意動態變更需求且完全不衍生額外成本。

Ch 2. 軟體開發流程 · 第 12 題

單元：增量 (Incremental) 與疊代 (Iterative)

在軟體流程工程中，「增量 (Incremental)」與「疊代 (Iterative)」開發的根本觀念差異是什麼？

- A 增量專注於自動化測試；疊代專注於 UI 介面視覺設計。
- B 增量分階段交付完整可運行的垂直功能切片；疊代在重複循環中深化打磨系統的整體初稿。**
- C 增量由專案經理負責主導；疊代完全由客戶端工程師自行決策。
- D 增量嚴格遵循瀑布規範；疊代完全不產生任何程式碼文檔。

Ch 2. 軟體開發流程 · 第 13 題

單元：最小可行產品 (MVP) 演進模式

在 Henrik Kniberg 著名的最小可行產品 (MVP) 隱喻中（從滑板到汽車），為什麼在第一期交付「一顆孤立的汽車輪子」被視為嚴重的反模式 (Anti-Pattern)？

- A 因為單獨製造一顆輪子的生產成本遠高於組裝滑板。
- B 因為單獨一顆輪子無法提供任何端到端的交通位移價值，使用者完全無法藉其驗證核心痛點。**
- C 因為在現代工業設計中汽車輪子無法直接升級為機車輪胎。
- D 因為軟體工程標準禁止團隊在第一期專案中開發圓形幾何物件。

Ch 2. 軟體開發流程 · 第 14 題

單元：敏捷宣言核心價值

敏捷宣言宣稱：*「可運行的軟體重於詳盡的文檔。」* 這項價值在實務工程中主要倡導何種行為？

- A 嚴格禁止工程團隊撰寫任何架構設計文件或 API 規格合約。
- B 雖然文檔具有其輔助價值，但團隊應將交付可點擊執行的軟體系統置於最優先級。**
- C 專案進度完全不需要撰寫程式碼，只需由客戶口頭認可即可。
- D 將所有軟體文檔直接替換為行銷部門的投影片。

Ch 2. 軟體開發流程 · 第 15 題

單元：敏捷原則之可持續開發步調

敏捷原則第八條指出：*「敏捷流程提倡可持續的開發步調。發起人、開發者和使用者應能長期維持恆常穩定的步調。」* 其最核心的工程動機為何？

- ☒ A 避免因長時間過勞導致代碼品質劣化、累積致命缺陷以及工程團隊耗竭流失。
- ☐ B 強制工程師每週必須提交超過一百次無意義的 Git Commit 記錄。
- ☐ C 限制開發團隊每天最多只能撰寫五十行以內的原始碼。
- ☐ D 確保軟體永遠不需要在生產環境中進行重大版本發行。

Ch 2. 軟體開發流程 · 第 16 題

單元：看板方法與半成品限制 (WIP Limit)

在看板 (Kanban) 流程框架中，對看板欄位強制設定嚴格的「在製品數量上限 (WIP Limits)」主要工程目的為何？

- A 限制工程師每日能夠修改的程式碼檔案總數。
- B 消除多工切換耗損、暴露出流程瓶頸，並迫使團隊在開啟新工作前先將既有任務推進至完成。**
- C 迫使軟體團隊必須恢復傳統瀑布模型的年度驗收測試。
- D 確保軟體專案的所有架構設計皆由外部顧問獨立完成。

Ch 2. 軟體開發流程 · 第 17 題

在 Scrum 框架中，每個衝刺末期召開的「衝刺回顧會議 (Sprint Retrospective)」其最主要的核心目標是什麼？

- A 向外界利益關係人正式演示可運行的軟體增量以獲取商業反饋。
- B 審視團隊內部的人際協同、工程流程與工具實踐，並制定具體的行動改善方案。**
- C 針對落後進度的工程師個人進行公開績效檢討與懲處。
- D 將整個軟體專案的所有產品待辦清單全面銷毀重來。

單元：衝刺回顧會議 (Sprint Retrospective)

Ch 2. 軟體開發流程 · 第 18 題

單元：軟體技術債與利息代價

依據 Ward Cunningham 提出的「技術債 (Technical Debt)」金融隱喻，下列何者代表軟體組織在未來所必須支付的「複利利息 (Interest)」？

- A 每年向公有雲端供應商支付的伺服器虛擬機器租賃費用。
- B 因架構混亂耦合與缺乏測試，導致後續每次新增或修改功能時額外耗費的摸索、除錯與修補代價。**
- C 發放給全體資深軟體架構師的年度專案分紅獎金。
- D 購買商業專利編譯器軟體授權的固定折舊費用。

Ch 2. 軟體開發流程 · 第 19 題

單元：疲軟 Scrum (Flaccid Scrum)

Martin Fowler 提出著名的**「疲軟 Scrum (Flaccid Scrum)」**一詞，是用來針砭軟體工程實務中的哪項重大失衡？

- ☒ A 全面導入 Scrum 的管理儀式與表象名詞，卻徹底拋棄了 TDD、持續重構與 CI 等技術工程實踐。
- ☐ B 拒絕使用昂貴的商業 Jira 專案追蹤軟體，堅持使用物理實體白板與紙本便利貼。
- ☐ C 允許產品負責人在衝刺規劃會議進行期間微調待辦清單的排序。
- ☐ D 強制在每一次 Git Commit 提交時必須由自動化 CI 流水線執行單元測試。

Ch 2. 軟體開發流程 · 第 20 題

單元：結對編程之領航員角色

在極限編程 (XP) 的結對編程 (Pair Programming) 實踐中，「領航員 (Navigator)」的最主要核心職責是什麼？

- A 專注於敲擊鍵盤語法並在本地終端機執行指令編譯。
- B 退後一步進行宏觀戰略思考，即時審閱代碼結構、推敲極端邊界條件並把關系統架構。**
- C 擔任專屬的 Scrum 敏捷教練並負責在 Jira 系統中拖動任務卡片。
- D 負責與客戶進行法律合約協商並審批團隊年度工程預算。

Ch 2. 軟體開發流程 · 第 21 題

單元：TDD 測試驅動開發與重構

在極限編程的「測試驅動開發 (TDD)」紅綠重構循環中，「重構 (Refactor)」階段的特定工程目標是什麼？

- A 為模組增添全新的業務功能並擴展其對外的公共 API 介面合約。
- B 在確保所有既有自動化測試持續維持綠燈通過的前提下，改善程式碼的內部架構、消除重複並提升可讀性。**
- C 徹底刪除所有執行時間超過一秒的自動化單元測試以加速 CI 構建。
- D 將應用系統從物件導向語言強行全盤改寫為純函式語言。

Ch 2. 軟體開發流程 · 第 22 題

單元：持續交付 (CD) vs. 持續部署

在現代 DevOps 實務中，「持續交付 (Continuous Delivery)」與「持續部署 (Continuous Deployment)」在實務維運上的根本決定性差異為何？

- A 持續交付需要仰賴人工編譯原始碼；持續部署則實現編譯自動化。
- B 持續交付在預發環境中暫停並需經由人工商業決策審批發行；持續部署則在通過全套自動化測試後直接自動推上生產環境。**
- C 持續交付專門用於行動 App 開發；持續部署專門用於關聯式資料庫。
- D 持續交付完全不需單元測試；持續部署強制要求結對編程。

Ch 2. 軟體開發流程 · 第 23 題

單元：AI 代理人與自動化驗證守門員

在現代「AI 規格驅動開發 (AI Specification-Driven)」實務中，自動化 CI/CD 驗證守門員的最主要核心角色是什麼？

- A 阻止人類開發者親自審閱人工智慧所生成的代碼產物。
- B 在 AI 生成的程式碼合入生產環境前，強制執行確定性品質檢查、測試合規性驗證與缺陷防堵。**
- C 將自然語言提示詞直接轉換為公有雲端供應商的月度扣款帳單。
- D 全面廢除傳統軟體規格書，改以未經驗證的龐雜聊天歷史紀錄替代。

第三章 需求工程

需求探索、規格契約、可量化指標與人機協同

「在建造軟體系統的所有環節中，最困難的單一任務，莫過於精準決定究竟要建造什麼。」 —— 佛瑞德·布魯克斯 (Fred Brooks)

”

Ch 3. 需求工程 · 第 24 題

單元：使用者需求 vs. 系統需求

在需求工程中，「使用者需求 (User Requirements)」與「系統需求 (System Requirements)」最關鍵的實務運作差異為何？

- A** 使用者需求是利害關係人的高階目標；系統需求則是供開發者實作的精確功能合約。
- B 使用者需求限定於 UI 線框圖；系統需求則專指後端資料庫的資料表綱要設計。
- C 使用者需求簽署後絕不可變更；系統需求則需在每日站立會議中持續進行重構。
- D 使用者需求來自外部法規稽核團隊；系統需求則由編譯器工具自動解析產生。

Ch 3. 需求工程 · 第 25 題

單元：領域需求 (Domain Requirements)

請參考 **UberEats** 外送平台的這段需求規格說明：> *「因應市政食品衛生安全法規，易腐熟食的外送運送時間不得超過 45 分鐘，且運送容器全程須維持在 60°C 以上。」*

請問上述敘述屬於哪一種軟體需求類別？

- A** 領域需求 (Domain Requirement，由運作環境、產業法規或物理定律所施加的強制約束)
- B** 功能性需求 (Functional Requirement，指定系統必須執行的主動計算、特定功能或服務)
- C** 非功能性需求 (Non-Functional Requirement，針對效能、擴充性或可用性等整體系統品質特性)
- D** 使用者需求 (User Requirement，由終端消費者所提出之高階自然語言願景或業務目標)

Ch 3. 需求工程 · 第 26 題

單元：可驗證的非功能性需求指標

客戶提出了一個模糊的非功能性目標：*「訂單結帳系統必須具備極致速度與極高可靠度。」* 下列哪一項敘述正確地將此模糊目標轉化為**可驗證、可測試的工程量化指標**？

- A** 99% 的結帳交易回應時間須 ≤ 500 毫秒，且尖峰時段系統可用性須達 $\geq 99.95\%$ 。
- B** 結帳使用者介面應採用滑順流暢的過場動畫，使顧客在心理層面感受最極致的速度。
- C** 所有結帳微服務皆須採用記憶體安全程式語言開發，以百分之百確保程式零缺陷執行。
- D** 雲端資料庫應於尖峰交易負載攀升時，自動配置無上限的隨機存取記憶體 (RAM)。

Ch 3. 需求工程 · 第 27 題

單元：訪談啟發技術與五個為什麼

在需求訪談過程中，一位醫院主管堅持要求：*「我們的臨床資訊系統必須導入區塊鏈帳本來記錄病患的生理生命徵象。」* 此時軟體工程師最應採用的專業訪談啟發法為何？

- A** 運用「五個為什麼 (5 Whys)」探詢其背後的資料防竄改與稽核痛點，而非過早鎖定特定技術。
- B** 立即針對客戶指定的區塊鏈功能，著手撰寫智慧合約與關聯式資料庫對應綱要。
- C** 當場直接駁回該主管的要求，因為非技術利害關係人不得主動指定系統的實作架構。
- D** 禮貌性終止本次訪談流程，並將後續需求擷取工作完全轉向被動的工作場所觀察法。

Ch 3. 需求工程 · 第 28 題

單元：民族誌觀察法與內隱知識挖掘

在需求工程中，當分析急診室或飛航管制等複雜且高壓的運作環境時，為何
民族誌觀察法 (Ethnography / 職場觀察) 具備無可取代的關鍵價值？

- A** 能發掘內隱知識——即使用者已習以為常且視為當然，因而在正式訪談中鮮少主動提及的工作細節與非正式捷徑。
- B** 能自動將利害關係人的自然語言需求，在完全無須人工介入的情況下直接編譯為可執行的驗收測試套件。
- C** 能完全消除後續的軟體架構設計、關聯式資料庫建模以及同儕程式碼審查 (Code Review) 等開發階段。
- D** 能確保所產出的軟體需求規格書在專案的第一個開發衝刺期 (Sprint) 即達成 100% 的數學完全性。

Ch 3. 需求工程 · 第 29 題

單元：使用案例圖與詳細說明書

軟體工程團隊繪製了一份 UML 使用案例圖，標註使用者與「提領現金」使用案例相連。為何工程師除了使用案例圖外，還必須撰寫詳盡的文字型**使用案例說明書 (Use Case Description)**？

- A 圖形僅呈現高階系統範疇；文字說明才能完整定義循序步驟、前置條件、後置條件與例外處理流程。
- B 現行現代網頁瀏覽器若缺乏伴隨的 Markdown 純文字描述，將完全無法正確渲染 UML 向量圖形。
- C 現代編譯器與虛擬機器需要文字型使用案例說明，以便在執行階段預先為角色執行緒配置堆積記憶體。
- D 文字型使用案例說明的核心目的，是將非功能性需求自動轉化為前端圖形使用者介面的高傳真原型。

Ch 3. 需求工程 · 第 30 題

單元：後期需求變更與骨牌效應代價

為何在軟體系統交付上線後才修正需求錯誤，其耗費的成本會遠高於在早期開發階段修正？

- A 需求規格文件一旦由客戶簽署核准後，在法律上便具有合約鎖定效力而嚴格禁止進行任何修改。
- B 後期修正必須耗費大量成本重新設計架構、重寫程式碼、重測並重新部署基於錯誤前提建立的連鎖元件。**
- C 雲端程式碼儲存庫的自動化編譯器，在軟體發布至正式生產環境後會自動鎖定並阻止程式碼變更。
- D 自動化單元測試套件在程式碼封裝並部署至雲端微服務叢集後，將會完全失去其工程驗證的有效性。

Ch 3. 需求工程 · 第 31 題

單元：生成式 AI 在需求工程中的幻覺風險

當軟體工程團隊運用大型語言模型 (LLM) 從利害關係人的訪談逐字稿中自動生成需求規格書時，最需要透過「人機協同 (Human-in-the-Loop)」進行驗證的核心風險是什麼？

- A LLM 可能產生聽起來合理但完全虛構的商業邏輯、遺漏關鍵的邊界約束，或捏造不存在的外部 API 串接。
- B LLM 完全無法產出以 Markdown 清單格式或標準使用者故事 Given-When-Then 語法結構編排的規格文字。
- C LLM 在生成文字時會消耗過量伺服器記憶體，進而導致正式環境中微服務叢集的資料庫發生死結錯誤。
- D LLM 嚴格遵循傳統瀑布式開發流程，因此會堅決拒絕為敏捷疊代衝刺週期生成增量式的需求項目。

恭喜！已完成全部 31 道核心觀念測驗

成功檢視軟體工程導論、流程模型與需求工程全貌

「日日行，不怕千萬里；常常做，不怕千萬事。」 —— 《格言聯璧》 „

複習第一章

回顧軟體危機、職責分離、ISO 25010 與工程倫理核心概念。

[跳轉至第 01 題 →](#)

複習第二章

回顧敏捷宣言、Scrum 儀式、看板 WIP 限制與 TDD 重構回圈。

[跳轉至第 10 題 →](#)

複習第三章

回顧需求獲取方法、可量化 NFR、使用案例與 AI 幻覺防範。

[跳轉至第 24 題 →](#)